
Table of Contents

Overview of the Programming Language.....	2
I. STRUCTURE OVERVIEW.....	3
II. GENERAL RULES.....	4
III. SPECIFIC RULES.....	11
DECLARATIONS.....	11
DATA TYPES AND LITERALS.....	12
IDENTIFIERS.....	18
VARIABLES.....	19
CONSTANTS.....	21
ARRAYS.....	22
STRUCTURES.....	25
OPERATORS.....	30
OPERATOR PRECEDENCE.....	34
EXPRESSIONS.....	35
STATEMENTS.....	40
FUNCTIONS.....	59
COMMENTS.....	66
IV. REGULAR DEFINITION.....	67
V. REGULAR EXPRESSION.....	69
VI. TRANSITION DIAGRAM.....	73
RESERVED WORDS.....	73
RESERVED SYMBOLS.....	76
IDENTIFIERS.....	80
COIN LITERALS.....	82
DIME LITERALS.....	84
PARCH AND SCROLL LITERALS.....	85
VII. CONTEXT FREE GRAMMAR.....	86
VIII. FIRST SET.....	110
IX. FOLLOW SET.....	120
X. PREDICT SET.....	130
XI. TEST SCRIPTS.....	145

Overview of the Programming Language

SeaStack is a high-level programming language designed to combine structured programming principles with a thematic and engaging syntax inspired by the ocean voyager theme. It takes its foundation from the C language, adapting familiar constructs such as functions, loops, conditionals, and structures while introducing an oceanic twist to make coding both intuitive and enjoyable.

The integration of the Ocean Voyager theme enhances the language by aligning programming commands with navigational and seafaring terms. This thematic consistency creates an immersive environment that not only strengthens recall of syntax but also sparks creativity in the programming experience.

SeaStack supports a wide range of constructs, including global and local declarations, expressions, statements, arrays, structures, and user-defined functions that enable programmers to implement both simple and complex logic. These features make SeaStack suitable for learning core programming concepts, developing simple yet structured algorithms, and introducing problem-solving skills more interactively.

Overall, the SeaStack programming language makes programming accessible and enjoyable by combining solid foundations with a creative oceanic theme. It encourages learners to see coding as a voyage of discovery, emphasizing clarity, engagement, and a fun learning experience.

I. STRUCTURE OVERVIEW

<global_declaration/s> | λ

LOCKE <DATA_TYPE> <id> = <value>!! ~ Global Constant | λ

<DATA_TYPE> <id>!! ~ Global Variable | λ

<DATA_TYPE> <id>{<size>}!! ~ Global Array | λ

MAST <id> [
 <member declaration/s>
]!! ~ Structure Definition | λ

<DATA_TYPE> <id>(<parameter/s>) [
 <local_declaration/s> | λ
 <statement/s> | λ
 BACK <return_value>!!
] ~ Returning Function | λ

ABYSS <id>(<parameter/s>) [
 <local_declaration/s> | λ
 <statement/s>
 BACK!! | λ
] ~ Non-returning Function | λ

~~ muti-line
comment ~~

AHOY() [
 <local_declaration/s> | λ
 ~ Main Program

<DATA_TYPE> <id>!! ~ Local Variable | λ

<DATA_TYPE> <id>{<size>}!! ~ Local Array | λ

MAST <id> <variable/s> !! ~ Structure Declaration | λ

<statement/s>

]

II. GENERAL RULES

1. SeaStack supports variables, constants, arrays, structures, expressions, statements, subfunctions, comments, and a single AHOY function. It also supports global and local declarations.
2. Global declarations include variables, constants, arrays, and structure and function definition. These are located at the start of the program and can be accessed or modified anywhere within the program.
3. Local declarations include variables, arrays, and structure variables, and it overrides global declarations of the same name and type. These are declared at the start of any function body and can only be accessed or modified within the function in which they are declared.
4. SeaStack uses two exclamation marks (!!) as a statement terminator.
5. The body of functions, structures, conditional statements and looping statements, as well as the initial values of an array and structure variable, must be enclosed within a pair of square brackets ([]).
6. SeaStack recognizes the following reserved words as data types: COIN, DIME, PARCH, SCROLL, and BOOL.
7. COIN is any whole number. It must contain at least one (1) digit and may contain up to sixteen (16) digits only. Leading zeros are not allowed.
8. DIME is any number with a decimal component. The integer part must contain at least one (1) digit and may contain up to sixteen (16) digits only. The decimal part may contain up to eight (8) digits. Leading zeroes, and trailing zeroes exceeding 8 digits, are not allowed.
9. PARCH is any single ASCII character, except newline and single quotation mark, or a single escape sequence, enclosed in a pair of single quotation marks (' ').
10. SCROLL is any sequence of ASCII characters, except newline and double quotation marks, enclosed in a pair of double quotation marks (" ").
11. BOOL can be either AYE (**true**) or NAY (**false**) only.
12. Identifiers are used to name variables, constants, arrays, structures, structure member or variable, and subfunctions. It must only start with a lowercase letter, and can be followed by a sequence of lowercase letters, numbers, and underscores only. Identifiers should have a length of at least one (1) and at most twenty (20) valid characters.
13. Variables are values that can be accessed and modified in the program. It is declared with the data type, then the identifier, and the statement terminator.

-
14. Constants are fixed values that can be accessed but cannot be modified in the program. It must be initialized at the time of declaration, beginning with the reserved word LOCKE, then the data type, identifier, assignment operator (=), the assigned value, and the statement terminator.
 15. Arrays are collections of elements that share the same data type. An array declaration begins with the data type, then the identifier, a fixed size enclosed in curly braces ({ }), and the statement terminator. Arrays in SeaStack can be either one-dimensional or two-dimensional only.
 16. Structures are groups of several related variables of different data types. It is defined after the declaration of any global constant, variable, or array, and before any function. It starts with the reserved word MAST, followed by the identifier, then the member declaration enclosed in square brackets, and ends with the statement terminator.
 17. A structure variable can only be declared inside the AHOY function. It must start with the reserved word MAST, followed by the structure member identifier, then the variable identifier, and end with the statement terminator.
 18. SeaStack supports arithmetic, assignment, unary, relational, logical, and concatenation operators. Operators follow the given operator precedence rule.
 19. Operators must not appear consecutively without any operand in between.
 20. SeaStack supports arithmetic, unary, relational, logical, and SCROLL expressions.
 21. Arithmetic expressions perform mathematical calculations on operands of type COIN or DIME only with arithmetic operators (+, -, *, /, %, ^).
 22. Unary expressions perform increment or decrement on operands of type COIN only with unary operators (+#, -#) as prefix.
 23. Relational expressions compare exactly two operands of type COIN or DIME only with less than (<), greater than (>), less than or equal (<=), or greater than or equal (>=) operators. Relational expressions using the operators equal to (==) or not equal to (!=) can compare two operands of any compatible data type.
 24. Logical expressions compare operands of type BOOL only with logical operators (!, !#, &&, ||).
 25. SCROLL expressions combine operands of type SCROLL with the concatenation operator (&).
 26. SeaStack supports assignment, input, output, conditional, looping, and jump statements.
 27. Assignment statements assign a value to the left-hand operand with the assignment operators (=, +=, -=, *=, /=, %=, ^=).

-
28. Input statements use the reserved word ASK to get user input, while output statements use the reserved word ECHO to display output.
 29. Conditional statements execute when a certain condition is met. LOOK, DROP, DROLOOK, and CHART are reserved words used for conditional statements. The body of conditional statements must be enclosed within square brackets.
 30. Looping statements execute repeatedly based on a given condition. HOIST, HEAVE, and HAUL-HEAVE are reserved words used for looping statements. The body of looping statements must be enclosed within square brackets.
 31. Jump statements immediately execute a block of code in another part of the program. LAND, SAIL, and BACK are reserved words used for jump statements.
 32. Returning functions must have a single BACK statement located at the end of the body. Statements, excluding the single BACK statement, are optional.
 33. Nonreturning functions and the AHOY function must have at least one statement, excluding jump statements.
 34. Subfunctions must be defined after the declaration of any global constant, variable, array, or structure definition, and before the AHOY function. It can either be a returning or nonreturning function. It must begin with a return type, followed by a unique identifier, then a pair of parentheses () with an optional parameter list inside, and the body enclosed within square brackets ([]).
 35. A returning function can only have a return type of COIN, DIME, PARCH, SCROLL, or BOOL and must end with a single BACK statement that returns a value matching the declared return type.
 36. A nonreturning function must only have a return type of ABYSS and must not return any value. It can end with a single BACK statement without a return value.
 37. The AHOY function serves as the main function, found at the end of the program. It must not have a return type, not include any parameter, and not have any BACK statement. It cannot be called by any function.
 38. Comments are ignored during execution. A single-line comment starts with a tilde symbol (~) followed by a sequence of characters, and ends with a newline.
 39. A multi-line comment starts with a tilde and open parenthesis (~(), and is enclosed with a closing parenthesis followed by a tilde ()~). A multi-line comment that is not enclosed is invalid.

RESERVED WORDS

Reserved Words	C Equivalent	Definition
ABYSS	void	Indicates that it will not return a value.
ADRIFT	default	Runs if no COURSE matches in a CHART statement.
AHOY	main	The starting or entry point of the program.
ASK	scanf	Reads input from the standard input stream.
AYE	true	Represents the value true.
BACK	return	Exits a function and optionally passes a value.
BOOL	bool	Used to store Boolean values, AYE (true) or NAY (false).
CHART	switch	Executes a block of code based on the value of an expression.
COIN	int	Represents whole numbers.
COURSE	case	Defines a branch inside a CHART statement.
DIME	double	Represents numbers with a decimal part.
DROP	else	Executes a block of code if the previous LOOK is false.
DROPLOOK	else if	Tests multiple conditions after an initial LOOK.

Reserved Words	C Equivalent	Definition
ECHO	printf	Prints output to the standard output stream.
HAUL-HEAVE	do-while	Creates a loop that executes once and loops while the condition is true.
HEAVE	while	Creates a loop that runs as long as a condition is true.
HOIST	for	Creates a loop that runs a fixed number of times.
LAND	break	Used to exit from a loop or CHART statement.
LOCKE	const	Defines an initial value that cannot be modified.
LOOK	if	Executes a block of code if the condition is true.
MAST	struct	Defines a collection of variables of different data types.
NAY	false	Represents the value false.
PARCH	char	Represents a single character.
SAIL	continue	Skips the loop and jumps to the next iteration.
SCROLL	string	Represents a sequence of characters.

RESERVED SYMBOLS

Operators	
Arithmetic Operators	$+$, $-$, $*$, $/$, $\%$, $^$
Assignment Operators	$=$, $+=$, $-=$, $*=$, $/=$, $\%=$, $\wedge=$
Unary Operators	$+\#$, $-\#$
Relational Operators	$<$, $>$, $<=$, $>=$, $==$, $!=$
Logical Operators	$!$, $!\#$, $ $, $\&\&$
Concatenation Operator	$\&$

Other Symbols	
RESERVED SYMBOLS	DEFINITION
@	Used as a prefix to reference a variable.
\$	Used to access the member of a structure variable.
,	Used to separate values.
:	Used to indicate the start of a COURSE's body.
{	Used to enclose the size or index of an array or the character of a SCROLL.
}	
(	Used to enclose parameters, arguments, grouped expressions, or conditions.
)	
[	Used to enclose a block of statements, initial values of an array, and member declaration or member initialization of a structure.
]	

Other Symbols	
RESERVED SYMBOLS	DEFINITION
'	Used to enclose a PARCH literal.
"	Used to enclose a SCROLL literal.
~	Used to start a single-line comment.
~(	Used to start a multi-line comment.
)~	Used to end a multi-line comment.
!!	Used as a statement terminator.

III. SPECIFIC RULES

DECLARATIONS

Establish the characteristics of variables, constants, arrays, or structures before they are used. SeaStack allows global and local declarations.

A. Global declarations - Located at the start of the program and can be accessed or modified anywhere within the program. Global declarations include the following:

1. Declaration or initialization of variables, arrays, or initialization of constants. Constants can only be declared globally. Local constant declarations are invalid.
2. Definition of structures
3. Definition of subfunctions

B. Local declarations - Located at the start of any function body and can only be accessed or modified within the function they are declared in. If a local and global declaration share the same identifier, the local declaration will override the global declaration. Local declarations include the following:

1. Declaration or initialization of variables or arrays
2. Declaration or initialization of structure variables

DATA TYPES AND LITERALS

Data types specify which type of value a variable can hold and the type of operations that can be performed on it. The reserved words COIN, DIME, PARCH, SCROLL, and BOOL are used as data types, while ABYSS is used as a return type for functions that do not return a value.

A. COIN - A whole number without a decimal component within a specified range.

Rules for COIN

1. A COIN literal represents whole numbers. The inclusion of any character other than digits or the dash sign (-) is not permitted.
2. A COIN accepts a maximum of sixteen (**16**) digits. The accepted range is from -9999999999999999 to 9999999999999999.
3. Leading zeros are not allowed unless the value is zero, in which case one zero remains.
4. Positive COIN literals are unsigned and represented without a prefix.
5. Negative COIN literals must be denoted by a dash sign (-) placed immediately before the first digit. Any whitespace between the dash sign and the digit is prohibited.

Examples

VALID	INVALID
20	20.5
1000000000000000	1,000,000,000,000,000
1234567890123456	12345678901234567
2000000000000001	002000000000000000
67	+67
-20	- 20
-34	3a4

B. DIME - A number with a decimal component within a specified range.

Rules for DIME

1. A DIME literal represents numbers with a decimal part. The inclusion of any character other than digits, the decimal point (.), or the dash sign (-) is not permitted.
2. A DIME accepts a maximum of sixteen (**16**) digits for the integer part and a maximum of eight (**8**) digits for the decimal part. The accepted range is from -9999999999999999.99999999 to 9999999999999999.99999999.
3. If the value has no integer part, a single leading zero is required before the decimal point. If no decimal part is provided, a decimal point followed by a single zero (**.0**) is automatically added to the value.
4. Leading zeros are not allowed unless they are the single leading zero required before the decimal point, or the value is zero. Any digit after the eighth digit in the decimal part is invalid.
5. Positive DIME literals are unsigned and represented without a prefix.
6. Negative DIME literals must be denoted by a dash sign (-) placed immediately before the first digit.

Examples

VALID	INVALID
0.12345678	000 .12345678
1.12345678	1.12345678 001
1234567890123456.12345678	1234567890123456 10 .12345678
12.5	12.5 a
0.123	. 123
20 → 20.0	20 .
-3.5	- 3.5

C. PARCH - A single character enclosed in single quotation marks (' ').

Rules for PARCH

1. PARCH represents a single ASCII character, except a newline, backslash, or single quotation mark, or a single escape sequence, enclosed in single quotation marks.
2. Using any symbol other than a pair of single quotation marks, or the lack thereof, is invalid to enclose a PARCH.
3. Any value enclosed in single quotation marks that exceeds or subceeds a single character or escape sequence is invalid.
4. PARCH recognizes only five escape sequences. These are valid only inside the pair of single quotation marks. Any instance of a backslash that is not followed by any valid escape sequence character is invalid.
 - a. `\s` : Single quote
 - b. `\n` : Newline
 - c. `\t` : Horizontal tab
 - d. `\0` : Null
 - e. `\\` : Backslash

Examples

VALID	INVALID
'A'	"A"
'+'	'++'
' '	"
'\n'	'\m'
'\s'	''
'\\'	'\\'

D. SCROLL - A sequence of characters enclosed in double quotation marks (" ").

Rules for SCROLL

1. SCROLL represents an ordered sequence of ASCII characters, except a newline, backslash, or double quotation mark, enclosed in double quotation marks.
2. Using any symbol other than a pair of double quotation marks, or the lack thereof, is invalid to enclose a SCROLL.
3. It must consist of at least one (**1**) valid ASCII character enclosed in double quotation marks.
4. Each character of a SCROLL is treated as a single-character SCROLL literal, and can be accessed using an index.
5. SCROLL recognizes only five escape sequences. These are valid only inside the pair of double quotation marks. Any instance of a backslash that is not followed by any valid escape sequence character is invalid.
 - a. **\d** : Double quote
 - b. **\n** : Newline
 - c. **\t** : Horizontal tab
 - d. **\0** : Null
 - e. **** : Backslash

Examples

VALID	INVALID
"Hello"	'Hello'
" "	""
"\d"	""
"CFarers\n"	"CFarers\ns"
"CFarers\n, AHOY!"	"CFarers\s, AHOY!"
"She said, \dHello World\d"	"She said, \dHello World"\d
"Line 1\nLine2"	"Line 1 Line2"

Rules for Accessing SCROLL

1. Individual characters in a SCROLL are accessed using an index operator. It must start with a literal or variable of type SCROLL, followed by the index enclosed in curly braces (`{ }`).
2. The index indicates the position of the character in the SCROLL and starts at zero (**0**). The index must be a literal or variable of type COIN only and must not be empty or exceed the length of the SCROLL.

Syntax

<code><SCROLL-literal>{<index>}</code>
<code><identifier>{<index>}</code>

Examples

VALID	INVALID
<code>word{1}</code>	<code>word</code>
<code>"Hello"{0}</code>	<code>"Hello"{}</code>
<code>message{i}</code>	<code>message{i+1}</code>
<code>"Hello"{1}</code>	<code>"Hello">{1.5}</code>
<code>name{index}</code>	<code>name{index()}</code>
<code>word{x}</code>	<code>word[2]</code>

E. **BOOL** - BOOL represents either of two values: AYE (true) or NAY (false).

Rules for BOOL

1. BOOL is restricted to only two values: AYE or NAY; otherwise, it is invalid.

Examples

VALID	INVALID
AYE	TRUE, True
NAY	FALSE, False
AYE	1
NAY	0
AYE	aye

F. **ABYSS** - Represents the absence of a value.

Rules for ABYSS

1. ABYSS is used as a return type for a nonreturning function to signify that the function will not return any value to the calling function.
2. It cannot be used to declare or call variables, constants, arrays, or structures.
3. The keyword ABYSS, as well as any function defined with it, cannot be used as operands or values inside any expression or assignment statement.

Examples

VALID	INVALID
ABYSS function() [ECHO("Hello")!!]	ABYSS function() !!
ABYSS run() [ECHO("Go")!!]	ABYSS count = 0 !!
	LOCKE ABYSS val = 0!!
	ABYSS arr{5} !!
	MAST item [ABYSS id!!]!!

IDENTIFIERS

Identifiers are designated names given to identify a variable, constant, array, structure, structure member or variable, or subfunction.

Rules in Naming Identifiers

1. An identifier must begin with a lowercase letter only. Subsequent characters may consist of a combination of lowercase letters, digits, and underscores only. The use of uppercase letters and any other special character is strictly prohibited.
2. It must consist of at least one (1) lowercase letter and at most twenty (20) valid characters.
3. An identifier must be unique within its defined scope. Variables, arrays, constants, structures, structure members, structure variables, and subfunctions within the same scope, must each have a unique identifier. Declaring or defining with an identical identifier is invalid.
4. SeaStack allows identifier shadowing. A local identifier may share the same name as a global identifier. In such cases, the local declaration will take precedence and override within its local scope.
5. Reserved words are strictly not allowed to be used as identifiers.

Examples

VALID	INVALID
number	N umber
age	_ age
person1	p ERSON 1
middle_name	middle_name \$
this_is_a_twentychar	this_is_a_twentyfour char
coin	COIN
COIN age!! DIME gwa!!	COIN age !! DIME age !!
COIN x!! AHOY() [DIME x!!]	AHOY() [COIN x !! DIME x !!]

VARIABLES

Variables are user-defined containers for storing values.

Rules in Declaring a Variable

1. Declaring a variable must begin with the data type, followed by the identifier, and end with the statement terminator (!!).
2. Multiple declarations of variables with the same data type are allowed and are separated with a comma (,).
3. Multiple variables with different data types cannot be declared at once.
4. Declared variables are undefined and cannot be used in the program until it is assigned a value.

Syntax

<code><data_type> <identifier>!!</code>
<code><data_type> <identifier1>, <identifier2>, ..., <identifierN>!!</code>

Examples

VALID	INVALID
COIN age!!	COIN age
DIME pi!!	pi DIME!!
SCROLL greet, welcome!!	SCROLL greet welcome !!
PARCH letter, digit!!	PARCH letter, COIN digit!!
BOOL flag!!	BOOL flag, AYE !!
BOOL power, level, flag!!	BOOL power && flag!!

Rules in Initializing a Variable

1. Initializing a variable must begin with the data type, followed by the identifier, the assignment operator (=), then the assigned value, and end with the statement terminator (!!).
2. The initial value assigned to a variable must be of the same data type as the variable. It can be a literal, variable, constant, array element, structure member, function return value, or expression.
3. Initialization and declaration of multiple variables with the same data type is allowed and is separated with a comma (,). Multiple variables with different data types cannot be initialized at once.

Syntax

<code><data_type> <identifier> = <value>!!</code>
<code><data_type> <identifier1> = <value>, ..., <identifierN> = <value>!!</code>
<code><data_type> <identifier1>=<value>, <identifier2>, ..., <identifierN>=<value>!!</code>

Examples

VALID	INVALID
COIN age = 9!!	COIN age = '9' !!
PARCH letter = x!!	PARCH letter = x
DIME sec = add(x,y), min = (2.2*3.3)!!	COIN sec = 1, DIME min = 2.2!!
SCROLL name = students{3}!!	SCROLL name = 1*2+3!!
BOOL flag = AYE, on = menu\$first!!	BOOL flag = AYE, on = 9!!
BOOL flag = AYE, choice, on = NAY!!	BOOL flag = AYE, choice, on = !!

CONSTANTS

Constants are fixed values. It must be given an initial value at the time of declaration, and this value must not be reassigned or modified after initialization.

Rules in Declaring and Initializing Constants

1. A constant must be initialized at the time of declaration; otherwise, it is invalid. It must begin with the reserved word `LOCKE`, followed by the data type, the identifier, the assignment operator (`=`), the assigned value, and end with the statement terminator (`!!`).
2. The value to be assigned to a constant must only be a literal matching the declared data type.
3. Any attempt to reassign a new value to a constant is invalid.
4. Initialization of multiple constants with the same data type is allowed and is separated with a comma (`,`). Multiple constants with different data types cannot be initialized at once.

Syntax

<code>LOCKE <data type> <identifier> = <value>!!</code>
<code>LOCKE <data_type> <identifier1> = <value>, ..., <identifierN> = <value>!!</code>

Examples

VALID	INVALID
<code>LOCKE DIME pi = 3.14!!</code>	<code>LOCKE DIME pi!!</code>
<code>LOCKE SCROLL name = "Juan"!!</code>	<code>LOCKE SCROLL name = 123!!</code>
<code>LOCKE BOOL flag = AYE!!</code>	<code>LOCKE BOOL flag = 10 > x!!</code>
<code>LOCKE SCROLL language = "English", programming_language = "SeaStack"!!</code>	<code>LOCKE SCROLL name = "Jose", nickname = names()!!</code>
<code>LOCKE COIN max_score = 100!!</code>	<code>LOCKE COIN max_score = 100!! max_score = 150!!</code>

ARRAYS

A fixed-size collection of elements of the same data type.

Rules for Arrays

1. The size of an array must be specified upon declaration with a COIN literal only. The size must be at least one (**1**).
2. The elements of an array can be a literal, variable, constant, array element, structure member, function return value, or expression, which evaluates to the declared data type of the array.
3. The number of elements assigned to an array must not exceed the declared size. If fewer elements are provided than the declared size, the remaining indices without a value will be assigned a value of null.

A. One-dimensional Array - A one-dimensional array is a collection of elements of the same data type in a single row or list of values.

Rules for One-dimensional Array

1. A one-dimensional array declaration begins with the data type, followed by the identifier, the size enclosed in curly braces (**{ }**), and the statement terminator (**!!**). Only one array can be declared or initialized at once.
2. It can be initialized with the assignment operator (**=**), followed by square brackets (**[]**) enclosing the array elements which are separated by commas (**,**), and end with the statement terminator (**!!**).

Syntax

<code><data_type> <identifier>{<size>}!!</code>
<code><data_type> <identifier>{<size>} = [<value1>, ..., <valueN>]!!</code>

Examples

VALID	INVALID
COIN numbers{2}!!	COIN numbers{ }!!
DIME average{3}!!	DIME arr1{5}, arr2{5}!!
PARCH letters{3} = ['A', 'B', 'C']!!	PARCH letters{3} = ['A', 'B', 'C', 'D']!!
BOOL flag{2} = [3<5, status(a, c)]!!	BOOL flag{2} = [AYE, 1]!!

B. Two-dimensional Array - A two-dimensional array is an array where each element is another array.

Rules for Two-dimensional Array

1. A two-dimensional array declaration begins with the data type, followed by the identifier, then the size of the row and the column, each enclosed in curly braces (`{ }`), and ends with the statement terminator (`!!`). Only one array can be declared or initialized at once.
2. It can be initialized using the assignment operator (`=`), with square brackets enclosing the elements (`[ ]`), and each element being another set of array elements, with each set enclosed in square brackets (`[ ]`).

Syntax

<code><data_type> <identifier>{<rows>}{<columns>}!!</code>
<code><data_type> <identifier>{<rows>}{<columns>} = [[<r1c1>, ..., <r1cN>], ..., [<rNc1>, ..., <rNcN>]]!!</code>

Examples

VALID	INVALID
DIME grades{2}{2}!!	DIME grades{2}{ } !!
PARCH letters{1}{2}!!	PARCH letters{1}{ x }!!
COIN table{1}{2} = [[1, add(x,y)]]!!	COIN table{1}{2} = [[1, 3 > 5]]!!
COIN table{2}{3} = [[1, 1, 1],[2, 2, 2]]!!	COIN table{2}{1} = [[1], [2], [3]]!!
COIN table{1}{3} = [[1, 2+3, 3]]!!	COIN table{ 1 , 3 } = [1, 2, 3]!!

Rules for Accessing Array Elements

1. Array elements are stored consecutively in an index starting from zero (**0**).
2. Accessing an array element starts with the array identifier, followed by the array index enclosed in curly braces. The index must be a literal or variable of type COIN only and must not be empty or less than zero.
3. Accessing array elements can be used to modify or assign a value to the specified index.

Syntax

<code><identifier>{<index>}</code>
<code><identifier>{<index>}{<index>}</code>

Examples

VALID	INVALID
<code>grade{3}</code>	<code>grade{}</code>
<code>nums{1}{2}</code>	<code>nums[1][2]</code>
<code>numbers{2}</code>	<code>numbers{2.5}</code>
<code>nums{x}</code>	<code>nums{get_num()}</code>
<code>table{i}{j}</code>	<code>table{i, j}</code>
<code>average{i}</code>	<code>average{COIN}</code>

STRUCTURES

A collection of several related variables of possibly different types. The items in a structure are called members, and they can be of any valid data type.

Rules for Defining Structures

1. Defining a structure begins with the reserved word **MAST**, followed by a unique identifier, then the member declarations enclosed within the square brackets ([]), and ends with the statement terminator (!!).
2. Declaring a member starts with the data type, followed by the identifier, and ends with the statement terminator (!!). There must be at least one member declaration in a structure definition.
3. Multiple members of the same data type can be declared at once by separating each member with a comma (,). Multiple members of different data types cannot be declared at once.

Syntax

<pre>MAST <identifier> [<data_type> <identifier>!!]!!</pre>
<pre>MAST <identifier> [<data_type> <identifier1>, <identifier2>, ..., <identifierN>!!]!!</pre>
<pre>MAST <identifier> [<data_type1> <identifier1>!! ... <data_typeN> <identifierN>!!]!!</pre>

Examples

VALID	INVALID
MAST person [SCROLL name!!]!!	person [SCROLL name!!]!!
MAST person [COIN age!!]!!	MAST person []!!
MAST person [COIN age, year!!]!!	MAST person [COIN age, DIME year!!]!!
MAST item [DIME price!!]!!	MAST item [DIME price]!!
MAST student [COIN age!! DIME gwa!!]!!	MAST student [COIN age!! DIME gwa!!]

Rules for Declaring Structure Variables

1. Declaring a structure variable must begin with the reserved word MAST, followed by the structure identifier, then the structure variable identifier, and the statement terminator (!!).
2. Multiple structure variables can be declared at once by separating each variable with a comma (,).

Syntax

MAST <structure_id> <variable_id>!!
MAST <structure_id> <variable1_id>, <variable2_id>, ..., <variableN_id>!!

Example

VALID	INVALID
MAST book book1!!	MAST book book1, MAST employee!!
MAST employee e1, e2, e3!!	MAST employee e (1+3)!!
MAST student s1, s2, s3!!	MAST S tudent s1!!
MAST book novel!!	MAST SCROLL book novel!!
MAST car c1, c2!!	MAST car c1 c2!!

Rules for Initializing Structure Variables

1. Initializing a structure variable must begin with the reserved word MAST, followed by the structure identifier, the structure variable identifier, the assignment operator (=), then the assigned values enclosed in square brackets ([]), and end with the statement terminator (!!).
2. Only one structure variable can be initialized at once.
3. The value assigned to the members of a structure variable can be a literal, variable, constant, array element, structure member, function return value, or expression, which evaluates to the declared data type of the member.
4. Values are assigned sequentially to the members in the order of their declaration in the structure definition. This sequence is maintained until the initialization block ends or a member operator is encountered.
5. To initialize a specific member, start with the member operator (\$), followed by the member identifier, then the assignment operator, and the assigned value. The succeeding value will be assigned to the next member, unless specified otherwise with another member operator.
6. A member can only be left uninitialized if all succeeding members are also uninitialized, unless a member operator is used to skip the order. Uninitialized members will be assigned a value of null.
7. Providing a number of initial values that exceeds the number of defined members is invalid.

Syntax

MAST <structure_id> <variable_id> = [<value>]!!
MAST <structure_id> <variable_id> = [<value1>, ..., <valueN>]!!
MAST <structure_id> <variable_id> = [\$<member_id> = <value>]!!
MAST <structure_id> <variable_id> = [\$<member_id1> = <value1>, ..., \$<member_idN> = <valueN>]!!

Example

VALID	INVALID
MAST vehicle truck = ["Foton", "12-wheeler"]!!	vehicle truck = ["Foton", "12-wheeler"]!!
MAST blk3 s1 = ["Liza", 19, 1.00]!!	MAST blk3 s1 = ["Liza"], s2 = ["Ann"] !!
MAST movies movie1 = [\$title = "Fantastic Mr. Fox"]!!	MAST movies movie1 = [title = "Fantastic Mr. Fox"]!!
MAST books book1 = [\$title = "Gone Girl", "Mystery", \$price = 499.99]!!	MAST books book1 = [\$title = "Gone Girl", "Mystery", COIN x!!]!!
MAST animal insect = ["spider", "8 legs"]!!	MAST animal reptile, mammal , insect = ["spider", "8 legs"]!!
	MAST animal insect = {"spider", "8 legs"}!!

Rules for Accessing Members

1. To access a member, it must start with the structure variable identifier, followed by the member operator (\$), and then the member identifier.
2. Accessing members can be used to modify or assign a value to the specific member.

Syntax

<code><variable_id>\$<member_id></code>

Example

VALID	INVALID
student1\$name	student1.name
employee1\$salary	employee1\$
student1\$age	student1\$MAST
employee1\$role	employee1()\$role
student1\$age = 20!!	MAST\$age = 20!!

OPERATORS

Operators are symbols that provide instructions on how to perform arithmetic, relational, and logical calculations, concatenate SCROLL values, or perform other special operations. SeaStack supports arithmetic, assignment, unary, relational, logical, and other operators.

Rules for Operators

1. Seastack operators follow a standard operator precedence table.
2. Operators must not be used consecutively without any operand in between, unless the second operator is the unary minus (-), logical not (!), or logical double not (!#).
3. A dash sign (-) that is immediately followed by a digit, without any whitespace in between, is always considered as a negative digit. Otherwise, it is treated as a subtraction or unary minus operator.
4. Arithmetic operators follow the PEMDAS order, where operations inside parentheses are evaluated first, followed by exponents, then multiplication, division, or modulo, and finally, addition or subtraction.
5. Logical operators are evaluated using short-circuit rules, meaning that in logical AND, if the first operand evaluates to false, the second operand is not evaluated, and in logical OR, if the first operand evaluates to true, the second operand is not evaluated.
6. The modulo operation with negative values preserves the sign of the dividend or left operand.

A. Arithmetic Operators - Used to perform basic mathematical operations.

OPERATOR	DEFINITION
+	Adds operands.
-	Subtracts operands.
*	Multiplies operands.
/	Divides operands.
%	Gets the remainder from dividing operands.
^	Exponentiates an operand.

B. Assignment Operators - Used to assign values or update existing values.

OPERATOR	DEFINITION
=	Assignment operator. Assigns the value of the right-operand to the left-operand.
+=	Add and Assign operator. It adds the right operand to the left operand and assigns the result to the left operand.
-=	Subtract and Assign operator. It subtracts the right operand from the left operand and assigns the result to the left operand.
*=	Multiply and Assign operator. It multiplies the left operand by the right operand and assigns the result to the left operand.
/=	Divide and Assign operator. It divides the left operand by the right operand and assigns the result to the left operand.
%=	Modulo and Assign operator. It divides the left operand by the right operand and assigns the remainder to the left operand.
^=	Exponent and Assign operator. Raises the left operand to the power of the right operand and assigns the result to the left operand.

C. Unary Operators - Used to perform an operation on a single operand.

OPERATOR	DEFINITION
+#	Used to increment an operand.
-#	Used to decrement an operand.
-	Used to negate an operand.

D. Relational Operators - Used to compare two values and returns a BOOL value.

OPERATOR	DEFINITION
<	Returns AYE if the left operand is less than the right operand. Otherwise, returns NAY.
>	Returns AYE if the left operand is greater than the right operand. Otherwise, returns NAY.
>=	Returns AYE if the left operand is greater than or equal to the right. Otherwise, returns NAY.
<=	Returns AYE if the left operand is less than or equal to the right. Otherwise, returns NAY.
==	Returns AYE if the left operand is equal to the right operand. Otherwise, it returns NAY.
!=	Returns AYE if the left operand is not equal to the right operand. Otherwise, returns NAY.

E. Logical Operators - Used to combine or negate bool expressions.

Logical Operators	
RESERVED SYMBOLS	DEFINITION
&&	Logical AND. Returns AYE if both operands evaluate to true. Otherwise, it returns NAY.
	Logical OR. Returns AYE if at least one operand evaluates to true. Otherwise, it returns NAY.
!	Logical NOT. Returns AYE if the operand evaluates to false, and NAY if it evaluates to true.
!#	Logical DOUBLE NOT. Returns AYE if the operand evaluates to true, and NAY if it evaluates to false.

F. Other Operators - Used to perform a specific function.

OPERATOR	DEFINITION
&	Concatenation operator. Used to concatenate values of type SCROLL.
@	Address operator. Used to access the address of an operand.
\$	Member operator. Used to access a member of a structure.
{}	Index operator. Used to access an element of an array or a character of a SCROLL literal.
()	Call operator. Used as a placeholder of arguments to be passed to a function call.

OPERATOR PRECEDENCE

Precedence	Operator	Operation	Associativity
1	()	Function Call, Grouping	Left to Right
	{ }	Index Accessing	
	\$	Member Accessing	
2	+#, -#	Increment, Decrement	Right to Left
	!, !#	Logical Not, Logical Double Not	
	-	Unary Minus	
3	^	Exponent	Right to Left
4	*, /, %	Multiply, Divide, Modulo	Left to Right
5	+, -, &	Add, Subtract, Concatenation	
6	<, >	Less than, Greater than	
	<=, >=	Less than or Equal, Greater than or Equal	
7	==, !=	Equality, Inequality	
8	&&	Logical And	
9		Logical Or	
10	=	Assignment	
	+=, -=, *=, /=, %=, ^=	Compound Assignment	

EXPRESSIONS

Expressions combine operands with operators to produce a single value.

A. Arithmetic Expressions - It performs mathematical calculations.

Rules for Arithmetic Expressions

1. An arithmetic expression must begin with an operand, followed by an arithmetic operator, and end with an operand.
2. An arithmetic expression must contain at least one arithmetic operator and two operands of type COIN or DIME.
3. Operands may be literals, variables, constants, array elements, structure members, function return values, or grouped expressions that evaluate to a COIN or DIME only.
4. A dash sign (-) preceding a digit, without any whitespace in between, is always considered as a negative digit. If the dash sign is not preceding a digit without any intervening whitespace, and is preceded by an operand, it is treated as a subtraction operator. Otherwise, it is treated as a unary minus operator.
5. A unary minus operator is used to negate a variable, constant, array element, structure member, function return value, or grouped arithmetic expression.

Syntax

<code><operand> <arithmetic operator> <operand></code>
<code><operand1> <arithmetic operator> <operand2> ... <arithmetic operator> <operandN></code>

Examples

VALID	INVALID
<code>2 * (x + 3)</code>	<code>5 +* 2</code>
<code>5- 3</code>	<code>5-3</code>
<code>items{5} * 10</code>	<code>items{5} + "USD"</code>
<code>(x * y + z) / (x - y)</code>	<code>(x * y + z) / ()</code>
<code>- x + 5.0</code>	<code>* x + 5.0</code>

B. Relational Expressions - It performs comparison of two operands.

Rules for Relational Expressions

1. A relational expression must begin with an operand, followed by a relational operator, and end with an operand.
2. A relational expression must contain exactly one relational operator and two operands.
3. For less than (<), greater than (>), less than or equal (<=), or greater than or equal (>=) operators, the operands must only be literals, variables, constants, array elements, structure members, function return values, arithmetic expressions, or grouped expressions that evaluate to a COIN or DIME only.
4. For equal to (==) and not equal to (!=) operators, the operands must only be literals, variables, constants, array elements, structure members, function return values, or grouped expressions. The operands must have the same data type, except for COIN and DIME values; otherwise, it is invalid. COIN and DIME values can be compared with each other using these operators.

Syntax

<code><operand> <relational operator> <operand></code>
--

Examples

VALID	INVALID
<code>(a + b) >= (a * b)</code>	<code>>= (a + b) (a * b)</code>
<code>5 < 20</code>	<code>5 < 10 < 20</code>
<code>4 <= (val * 2)</code>	<code>username > "admin"</code>
<code>1 == 1.5</code>	<code>"Hello" == AYE</code>
<code>val >= flag</code>	<code>val >> flag</code>

C. Logical Expressions - It performs comparison of BOOL operands.

Rules for Logical Expressions

1. For the logical not (!) and logical double not (!#) operators, it must start with exactly one operator followed by a single operand.
2. For the logical or (||) and logical and (&&) operators, it must start with an operand, followed by the logical operator, and end with an operand.
3. A logical expression must contain at least one logical operator, and its operands must only be literals, variables, constants, array elements, structure members, function return values, relational expressions, or grouped expressions that evaluate to a BOOL.

Syntax

<operand> <logical operator> <operand>
<operand1> <logical operator> <operand2> ... <logical operator> <operandN>
!<operand>
!#<operand>

Examples

VALID	INVALID
num1 > 0 && num2 < 10	&& (num1 > 0)
!NAY	AYE ! NAY
count > 5 count < 100	(count > 5) (count < 100)
3 + 5 > 6 && 4 == 3	3 + 5 > 6 && 4
x > 5 && y < 10 !! 3 == 6	(x > 5) &&&
a && b c && d	a && b c && d

D. Unary Expressions - It performs increment or decrement.

Rules for Unary Expressions

1. A unary expression must start with the unary operator followed by a single operand. Any whitespace between the operator and the operand is invalid.
2. The operand must be a variable, array element, or structure member of type COIN only.
3. A unary expression may appear either as part of a condition in a HOIST statement or as a standalone statement ending with the statement terminator (!!).

Syntax

<code><unary operator><operand></code>
--

Examples

VALID	INVALID
<code>-#x</code>	<code>-# x</code>
<code>+#count</code>	<code>++count</code>
<code>-#value</code>	<code>value-#</code>
<code>+#age{3}</code>	<code>+#(5+3)</code>
<code>-#flag\$count</code>	<code>+#9</code>

E. SCROLL Expressions - A combination of SCROLL values combined with the concatenation operator.

Rules for SCROLL Expressions

1. A SCROLL expression must begin with an operand, followed by the concatenation operator (**&**), and end with an operand.
2. Operands can be literals, variables, constants, array elements, structure members, function return values, grouped SCROLL expressions, or SCROLL expressions of type SCROLL.
3. SCROLL expressions are evaluated from left to right and return a single SCROLL value formed by appending the operands in order without altering any character.

Syntax

<operand> & <operand>
<operand1> & <operand2> ... & <operandN>

Examples

VALID	INVALID
"Hi" & "There"	"Hi" "There"
"Name: "&name1&" "&name2	"Name: " & name1 name2
("Good" & "Morning") & "Boss"	"Good & "Morning" & "Boss"
("Sea" & "Stack") & " " & "Year" & " " & "2026"	("Sea" & "Stack") && "Year" & "" & "2026"
"Hello" & "There"	"Hello" + "There"
greeting & " Boss"	greeting & !!

STATEMENTS

Statements are instructions that the computer executes. SeaStack supports input, output, assignment, conditional, looping, and jump statements.

A. Input Statements - Used to obtain user input.

Rules for Input Statements

1. It must start with the reserved word **ASK**, followed by the argument list enclosed in parentheses, and end with the statement terminator (**!!**).
2. The first argument consists of the format specifier/s only enclosed in double quotation marks, followed by a comma, then the second argument which is the addressing of target variable, array element, or structure member.
3. Format specifiers must begin with a percentage symbol (**%**), followed by the first character of the data type (**%C**: COIN, **%D**: DIME, **%P**: PARCH, **%S**: SCROLL, **%B**: BOOL). It must strictly match the data type, quantity, and order of the targets in the second argument, and must not be separated by any comma.
4. Addressing the target variable, array element, or structure member starts with the address symbol (**@**) followed by the identifier. Multiple target variables must be separated with a comma.

Syntax

<code>ASK("<format_specifier>", @<identifier>)!!</code>
<code>ASK("<format_specifier1><format_specifier2>...<format_specifierN>", @<identifier1>, @<identifier2>, ..., @<identifierN>)!!</code>

Examples

VALID	INVALID
<code>ASK("%C", @score)!!</code>	<code>ASK("%C", score)!!</code>
<code>ASK("%D%C", @gwa, @score)!!</code>	<code>ASK("%C%D", @cost)!!</code>
<code>ASK("%C%D%D", @age, @weight, @height)!!</code>	<code>ASK("%s", @name)!!</code>

ASK("%C%C", @score, @total)!!	ASK("%C, %C", @score, @total)!!
ASK("%D", @gwa)!!	ASK("%D" @gwa)!!
ASK("%C", @player\$hp)!!	ASK("%C", @player.hp)!!

B. Output Statements - Used to display an output.

Rules for Output Statements

1. It must start with the reserved word ECHO, followed by the argument list enclosed in parentheses, and end with the statement terminator (!!).
2. The first argument is the text to be displayed and must be a SCROLL literal. It can be combined with format specifier/s which must be placed within the SCROLL literal.
3. A format specifier is used as a placeholder of a value to be displayed. If the first argument does not have any format specifier, it must not have a second argument.
4. The value/s in the second argument can be a literal, variable, constant, array element, structure member, function return value, or expression. The value/s must strictly match the data type, quantity, and order of the format specifier/s in the SCROLL literal. Multiple values must be separated with commas.

Syntax

ECHO("<SCROLL-literal>")!!
ECHO("<SCROLL-literal>", <value>)!!

Examples

VALID	INVALID
ECHO("Enter your age: ")!!	ECHO(Enter your age)!!
ECHO("Your score: %C", score)!!	ECHO("Your score: %C")!!
ECHO("Hello\nWorld")!!	ECHO("Hello" & greet)!!

ECHO("Your name: %S", name)!!	ECHO("Your name: name", %S)!!
ECHO("%C+%C = %C", a, b, sum)!!	ECHO("%C+%C = %C", a, b, sum)

C. Assignment Statements - Used to assign a value to the operand.

Rules for Assignment Statements

1. It must start with an operand, then the assignment operator, the value to be assigned, and end with the statement terminator (!!). The operand must only be a declared variable, array element, or structure member.
2. For the standard assignment operator (=), the operand can be any data type, and the value to be assigned must be a literal, variable, constant, array element, structure member, function return value, expression, or grouped expression of the same data type as the operand.
3. For the compound assignment operators (+=, -=, *=, /=, %=, ^=), the operand must be of type COIN or DIME. The value to be assigned must be a literal, variable, constant, array element, structure member, function return value, or arithmetic expression that evaluate to a COIN or DIME.

Syntax

<operand> <assignment_operator> <value>!!

Examples

VALID	INVALID
num1 = 10*2!!	num1 = 10
flag = AYE && 3 > (5+4)!!	9 = num2!!
username = "captain_01"!!	username() = "hi"!!
total *= 9!!	total * 9 = 5!!
height *= 3+5/(4-6)!!	height *= "tall"!!

D. Conditional Statements - Used to execute different blocks of code based on a given condition. The reserved words LOOK, DROP, DROPLOOK, and CHART are used for conditional statements.

a. LOOK - Executes the following block of statements if the given condition is true; otherwise, the block of statements is ignored.

Rules in LOOK Statements

1. It must start with the reserved word LOOK, then a condition enclosed in parentheses (()), then the body enclosed in square brackets ([]).
2. The condition must be a literal, variable, constant, array element, structure member, function return value, relational expression, or grouped expression that evaluates to a BOOL data type. It must not be empty.
3. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.

Syntax

LOOK (<condition>) [<statement/s>]
LOOK (<condition>) []

Examples

VALID	INVALID
LOOK (b > 5) [ECHO("b equals 5")!!]	LOOK (COIN b = 5) [ECHO("Invalid condition")!!]
LOOK (AYE) []	LOOK (AYE) []

ECHO("Always true")!! SAIL!!]	SAIL!! ECHO("Always true")!!]
LOOK (3+5>6 4 < 5) []	LOOK (x = 10) []
LOOK (is_valid == AYE) [ECHO("Granted")!!]	LOOK () [ECHO("Empty")!!]
LOOK (score >= 90) [ECHO("Excellent")!!]	LOOK [ECHO("True")!!]

b. DROP - Executes the block of statements under DROP only if the preceding LOOK or DROLOOK conditions does not evaluate to true.

Rules in DROP Statements

1. A DROP statement must have an initial LOOK statement and can be preceded by a DROLOOK statement.
2. The reserved word DROP must be followed by the body enclosed in square brackets ([]).
3. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.

Syntax

LOOK (<condition>) [<statement/s>] DROP [<statement/s>]
LOOK (<condition>) [] DROP []

Examples

VALID	INVALID
LOOK (is_rainy) [] DROP []	LOOK (x > 5) [ECHO("x is greater")!!] DROP (x<5) []
LOOK (age >= 18) [ECHO("You are old.\n")!!] DROP [ECHO("You are not old.\n")!!]	LOOK (age >= 18) [] DROP [SAIL!! LAND!!]
LOOK (flag) [] DROP [ECHO("ON.")!!]	LOOK (is_safe) [] DROP [BACK !!]
LOOK (flag) [DROP [ECHO("ON.")!!]	DROP [ECHO("ON.")!!]
LOOK (is_rainy) [] DROP []	LOOK (is_rainy) [] DROP ECHO("No Brackets")!!

c. DROPLOOK - Evaluates the conditions from top to bottom. If the condition does not evaluate to true, it proceeds to check the next condition. Once a condition is evaluated to be true, it executes the corresponding block of statements and skips the rest of the conditions.

Rules in DROPLOOK Statements

1. A DROPLOOK statement must have an initial LOOK statement and can be preceded by another DROPLOOK statement. It can be followed by another DROPLOOK statement or an optional final DROP statement.
2. The reserved word DROPLOOK must be followed by a condition enclosed in parentheses (()), then the body enclosed in square brackets ([]).

3. The condition must be a literal, variable, constant, array element, structure member, function return value, relational expression, or grouped expression that evaluates to a BOOL data type. It must not be empty.
4. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.

Syntax

```
LOOK (<condition1>) [  
    <statement/s>  
] DROPLOOK (<condition2>) [  
    <statement/s>  
] ... DROPLOOK (<conditionN>) [  
    <statement/s>  
]
```

```
LOOK (<condition1>) [  
    <statement/s>  
] DROPLOOK (<condition2>) [  
    <statement/s>  
] ... DROPLOOK (<conditionN>) [  
    <statement/s>  
] DROP [  
    <statement/s>  
]
```

```
LOOK (<condition1>) [  
] DROPLOOK (<condition2>) [  
] DROP [  
]
```

Examples

VALID	INVALID
<pre>LOOK (score >= 90) [ECHO("Excellent.")!!] DROPLOOK (score >= 75) [ECHO("Passed.")!!] DROP [ECHO("Failed.")!!]</pre>	<pre>DROPLOOK (score >= 75) [ECHO("Missing LOOK")!!] DROP [ECHO("Failed.")!!]</pre>
<pre>LOOK (temp > 37.5) [ECHO("Very hot")!!] DROPLOOK (temp > 30.5) [ECHO("Hot")!!] DROPLOOK (temp > 20.5) [ECHO("Warm")!!]</pre>	<pre>LOOK () [ECHO("No Condition")!!] DROPLOOK () [ECHO("No Condition")!!] DROPLOOK () [ECHO("No Condition")!!]</pre>
<pre>LOOK (grade == 'A') [ECHO("Outstanding")!!] DROPLOOK (grade == 'B') [ECHO("Good")!!] LAND!!]</pre>	<pre>LOOK (grade == 'A') [ECHO("Outstanding")!!] DROPLOOK (grade == 'B') [ECHO("Good")!!] BACK!!]</pre>
<pre>LOOK (condition{0}) [ECHO("True")!!] DROPLOOK (condition) [ECHO("False")!!] DROP []</pre>	<pre>LOOK (3+6) [ECHO("Not Boolean")!!] DROPLOOK () [ECHO("No Condition")!!] DROP []</pre>
<pre>LOOK (temp > 100) [ECHO("Boiling")!!] DROPLOOK (temp > 0 && temp <= 100) [ECHO("Liquid")!!]</pre>	<pre>LOOK (temp > 100) [ECHO("Boiling")!!] DROPLOOK (COIN x) [ECHO("Wrong Cndtn")!!]</pre>

d. Chart Statements - Executes different parts of the code based on the value of a single variable.

Rules in CHART Statements

1. It must start with the keyword CHART, followed by a condition enclosed in parentheses (()), and end with the body enclosed in square brackets ([]).
2. The condition of a CHART statement must be a literal or variable of type COIN, PARCH, or SCROLL only and must not be empty.
3. The body of a CHART statement must consist of at least one COURSE body and can end with a single optional ADRIFT body.
4. Each COURSE body must start with the keyword COURSE, followed by a unique label, then a colon symbol (:), followed by the statements to be executed. The label must be a COIN, PARCH, or SCROLL literal. Accessing a SCROLL character is allowed given that the index is a COIN literal that is within the length of the SCROLL.
5. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.
6. The optional ADRIFT body must start with the keyword ADRIFT, followed by a colon (:), then the statements to be executed. It supports all statements, except BACK and SAIL statements, and must have a LAND statement at the end.

Syntax

```
CHART (condition) [  
  COURSE label1:  
    <statement/s>  
]
```

```
CHART (condition) [  
  COURSE label1:  
    <statement/s>  
  ... COURSE labelN:  
    <statement/s>  
  ADRIFT:  
    <statement/s>  
  LAND!!  
]
```



```
CHART (condition) [  
  COURSE label1:  
]
```

Examples

VALID	INVALID
<pre>CHART (day) [COURSE 1: ECHO("Monday\n")!! LAND!!]</pre>	<pre>CHART (night, day) [COURSE 1: ECHO("Invalid Condition")!! LAND!!]</pre>
<pre>CHART (display) [COURSE 1: ECHO("Hello")!!]</pre>	<pre>CHART (letter) [COURSE a: ECHO("Variable as label")!! LAND!!]</pre>
<pre>CHART (menu) [COURSE 1: ECHO("File Menu")!! SAIL!!]</pre>	<pre>CHART (menu) [ADRIFT: ECHO("No COURSE body")!! LAND!!]</pre>
<pre>CHART (letter) [COURSE 'A': ECHO("Excellent")!! LAND!! ADRIFT: ECHO("Below Average")!! LAND!!]</pre>	<pre>CHART (digit) [COURSE 3: ECHO("Variable as label")!! LAND!! ADRIFT: ECHO("Variable as label")!! SAIL!!]</pre>
<pre>CHART (grade) [COURSE 'A': ECHO("Pass")!! LAND!! ADRIFT: LAND!!]</pre>	<pre>CHART (grade) [COURSE 'A': ECHO("Pass")!! LAND!! ADRIFT: BACK!]</pre>

E. Looping Statements - A looping statement executes a block of statements repeatedly until a given condition is satisfied. The reserved words HOIST, HEAVE, and HAUL-HEAVE are used for looping statements.

a. HOIST - Iterates over a specific range of numbers.

Rules in HOIST Statement

1. It must always begin with a reserved word HOIST, followed by the condition enclosed in parentheses (()), and end with the body enclosed in square brackets ([]). The HOIST condition has three arguments: initialization, condition, and update.
2. The initialization argument defines the starting state of the loop. The value assigned in the initialization must be a literal, variable, constant, array element, structure member, function return value, or arithmetic expression evaluating to type COIN only. It can be any of the three ways:
 - a. Initialization of a COIN variable starting with the reserved word COIN, followed by the identifier, the assignment operator (=), the value, and the statement terminator (!!). Multiple initialization is allowed and must be separated with commas.
 - b. Assignment of a COIN value starting with the operand (variable, array element, or structure member), followed by the assignment operator, the value, and the statement terminator. Multiple assignments are allowed and must be separated with commas.
 - c. A lone statement terminator.
3. The condition is checked before each iteration. If the condition evaluates to NAY (false), the loop terminates; otherwise, the loop continues. The condition must be a relational expression wherein the operands can only be literals, variables, constants, array elements, structure members, function return values, or arithmetic expressions of type COIN or DIME, and it must not be empty. Multiple relational expressions are allowed and must strictly be separated with logical operators.
4. The update argument runs after each iteration before re-evaluating the condition. It supports increment or decrement through a unary expression or assignment of values using a compound assignment operator. Multiple updates are allowed and must be separated with commas.

5. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.

Syntax

HOIST (<initialization>!! <condition>!! <update>) [<statement/s>]
HOIST (<initialization>!! <condition>!! <update>) []
For Initialization
<data_type> <variable_id> = <value>!!
<identifier> = <value>!!
!!
For Condition
<operand><relational_operator><operand>!!
<relational_expression><logical_operator><relational_expression>!!
For Update
<unary_op><identifier>
<identifier> <compound_assign_op> <value>

Examples

VALID	INVALID
HOIST (COIN i = 0!! i < 5!! +#i) []	HOIST (COIN i = 0 i < 5 i++) []
HOIST (i{4} = 0!! i{4}<5!! +#i{4}) [ECHO("i = %C, j = %C\n", i, j)!!]	HOIST (COIN i = 0!! i--) [ECHO("No condition")!!]

HOIST (!! x > 0!! x+=1) [ECHO("Countdown: %C\n", x)!!]	HOIST() [ECHO("empty condition")!!]
HOIST (COIN i = 0, j = 10!! i < j!! +#i, -#j) []	HOIST (COIN i = 0, x = "start"!! i < j!! +#i, -#j) []
HOIST (COIN i = 0!! i < 10!! +#i) []	HOIST (COIN i = 0!! i < 10!! +#i) [BACK!!]
HOIST (COIN i = 0, j = 10!! i < 10!! +#i, +#j) [ECHO("i is %C, j is %C\n", i, j)!!]	HOIST (COIN i = 0, j = 10!! i < 10!! +#i+#j) [ECHO("i is %C, j is %C\n", i, j)!!]
HOIST (COIN i = 0 !! i < 10 && signal == 1 !! +#i) [ECHO("The signal is %C", signal)!!]	HOIST (COIN i = 0!! i < 10!! +#i) [BACK!!]
HOIST (COIN i = 1 !! i + 1 < 10 && flag == 1 !! +#i) [ECHO("Iteration: %C\n", i)!!]	HOIST (COIN i = 0, i < 10, +#i) []

b. HEAVE - Executes the block of statements repeatedly while the given condition evaluates to true.

Rules in HEAVE Statements

1. It must always begin with the reserved word HEAVE, followed by the condition enclosed in parentheses (()), then the body enclosed in square brackets ([]).
2. The condition must be a literal, variable, constant, array element, structure member, function return value, relational expression, or grouped expression that evaluates to a BOOL data type. It must not be empty.
3. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.

Syntax

<pre>HEAVE (<condition>) [<statement/s>]</pre>
<pre>HEAVE (<condition>) []</pre>

Examples

VALID	INVALID
<pre>HEAVE (i < 5) [ECHO("i = %C\n", i)!! +#i!!]</pre>	<pre>HEAVE (i < 5)!! [i++!!]</pre>
<pre>HEAVE (choice != 3) [ECHO("Enter 3 to exit: ")!! ASK("%C", @choice)!!]</pre>	<pre>HEAVE choice !=3 [ECHO("Enter 3 to exit: ")!! ASK("%C", @choice)]</pre>
<pre>HEAVE (count > 0) [ECHO("Count: %C\n", count)!! -#count!!]</pre>	<pre>HEAVE () [ECHO("Empty condition")!!]</pre>
<pre>HEAVE (player\$hp > 0) []</pre>	<pre>HEAVE (100) []</pre>
<pre>HEAVE (fuel > 0) [ECHO("Flying...")!! fuel -= 10 !! SAIL !!]</pre>	<pre>HEAVE (fuel > 0) [ECHO("Flying...")!! fuel -= 10 !! BACK!!]</pre>

c. HAUL-HEAVE - Executes the block of statements at least once, then executes it repeatedly as long as the condition is AYE (**true**), and terminates once the condition evaluates to NAY (**false**).

Rules in HAUL-HEAVE Statements

1. The HAUL-HEAVE statement begins with the HAUL keyword, followed by the HAUL body enclosed in square brackets ([]).
2. The HAUL body must be followed by the HEAVE keyword, a condition enclosed in parentheses (()), and end with the statement terminator (!!).
3. The condition must be a literal, variable, constant, array element, structure member, function return value, relational expression, or grouped expression that evaluates to a BOOL data type. It must not be empty.
4. The body supports all statements, except BACK statements, and can be empty. LAND and SAIL statements are optional and the body can only have either one located at the end.

Syntax

<pre>HAUL [<statement/s>] HEAVE (<condition>)!!</pre>
<pre>HAUL [] HEAVE (<condition>)!!</pre>

Examples

VALID	INVALID
<pre>HAUL [ECHO("i = %C\n", i)!! +#i!!] HEAVE (i < 5)!!</pre>	<pre>HAUL [ECHO("No condition")!!] HEAVE!!</pre>
<pre>HAUL [+#x!!] HEAVE (x < 10)!!</pre>	<pre>HAUL [+#x!!] HEAVE (x + 10)!!</pre>

HAUL [ECHO("Processing...")!! SAIL !!] HEAVE (AYE)!!	HAUL [BACK !!] HEAVE (AYE)!!
HAUL [ECHO("Processing...")!!] HEAVE (tries < 4)!!	HAUL [ECHO("Processing...")!!] HEAVE [tries < 4]!!
HAUL [ECHO("Processing...")!!] HEAVE (is_waiting)!!	HAUL [ECHO("Processing...")!!] HEAVE (is_waiting)

F. Jump Statements - A jump statement immediately jumps to execute a block of code in another part of the program.

a. LAND Statement - It uses the reserved word LAND to immediately exit or terminate a conditional or looping statement.

Rules for LAND Statements

1. The LAND statement must begin with the reserved word LAND and end with the statement terminator (!!).
2. It is required to end an ADRIFT body. For other conditional and looping statements, it is optional and can only be placed at the end of the body. Otherwise, it is considered invalid.

Syntax

LAND!!

Examples

VALID	INVALID
LAND!!	LAND(1<3)!!
	Land !!
AHOY()[LOOK(3>5) [LAND!!	AHOY() [LAND!!]

<pre>]] </pre>	
<pre> AHOY() [CHART (day) [COURSE 1: ECHO("Monday")!! LAND!! ADRIFT: ECHO("Noramal Day")!! LAND!!]] </pre>	<pre> AHOY()[LOOK(3>5) [] LAND!!] </pre>
	<pre> AHOY() [LOOK (3 > 5) [LAND!! ECHO("Land must be last")!!]] </pre>

b. SAIL Statement - It uses the reserved word SAIL to skip the rest of the current iteration and continue to the next iteration.

Rules for SAIL Statements

1. The SAIL statement must begin with the reserved word SAIL and end with the statement terminator (!!).
2. It must not be used in an ADRIFT body. For other conditional and looping statements, it is optional and can only be placed at the end of the body. Otherwise, it is considered invalid.

Syntax

SAIL!!

Examples

VALID	INVALID
SAIL!!	SAIL id !!
	Sail !!
<pre> AHOY()[LOOK(3>5) [SAIL!!]] </pre>	<pre> AHOY()[LOOK(3>5) [] SAIL!!] </pre>

]	]
<pre>LOOK(x > y) [ECHO("Correct placement")!! SAIL!!]</pre>	<pre>AHOY() [LOOK(3 > 5) [SAIL!! ECHO("Sail must be last")!!]]</pre>
<pre>AHOY() [CHART(anchor) [COURSE 1: ADRIFF: ECHO("This is valid.")!! LAND!!]]</pre>	<pre>AHOY() [CHART(anchor) [COURSE 1: ADRIFF: SAIL!! LAND!!]]</pre>

c. BACK Statement - It uses the reserved word BACK to terminate the execution of a function. It may be used to return a value to the calling function.

Rules for BACK Statements

1. A BACK statement must begin with the reserved word BACK, followed by an optional return value, and end with the statement terminator (!!).
2. For returning functions:
 - a. A BACK statement is mandatory at the end of the body.
 - b. The keyword BACK must be followed by the return value. It must be a literal, variable, constant, array element, structure member, function return value, or expression of type COIN, DIME, BOOL, PARCH, or SCROLL only, and end with a statement terminator (!!).
 - c. The data type of the returned value must match the function's declared return type.
3. For non-returning functions:
 - a. A BACK statement is optional.
 - b. The keyword BACK must be followed by the statement terminator (!!). Indicating a return value is invalid.
4. For AHOY function:
 - a. There must be no BACK statement.

Syntax

BACK <value>!!
BACK!!

Examples

VALID	INVALID
BACK a!!	BACK ECHO("Invalid")!!
BACK!!	Back!!
BACK!!	AHOY()[BACK!!]
COIN add() [BACK a + b!!]	COIN add() [BACK ECHO("No")!!]
ABYSS display() [ECHO("Hi")!! BACK!!]	ABYSS display() [ECHO("Hi")!! BACK 10!!]

FUNCTIONS

A function is a self-contained block of code designed to perform a specific task or related set of tasks. Functions may or may not return values depending on their intended purpose. The types of functions in SeaStack are the main function, returning functions, and non-returning functions, each serving distinct roles in program execution.

A. AHOY Function - The main function uses the reserved word AHOY and is required in every program. This function coordinates the flow of instructions by calling other functions and managing the program's control structure.

Rules for AHOY Function

1. A valid SeaStack program must have exactly one AHOY function. It must be located at the bottom of the program.
2. It starts with the reserved word AHOY followed by a pair of parentheses (`()`), and ends with the body enclosed in square brackets (`[ ]`).
3. The AHOY function cannot accept arguments, hence the parentheses must be left empty.
4. The body must have at least one statement, excluding jump statements, and must not have any BACK statement. Local declarations are optional and located at the top of the body, before any statement.

Syntax

<pre>AHOY() [<statement/s>]</pre>
<pre>AHOY() [<local_declaration/s> <statement/s>]</pre>

Examples

VALID	INVALID
<pre>AHOY() [COIN age = 18!! ECHO("Age: %C", age)!!]</pre>	<pre>COIN AHOY() [COIN x = 10!! BACK x!!]</pre>
<pre>AHOY() [COIN x = 10!! ECHO("X: %C\n", x)!!]</pre>	<pre>AHOY(x) [COIN x = 10!! ECHO("X: %C\n", x)!!]</pre>
<pre>AHOY() [COIN x = 5, y = 10, z = x + y!! ECHO("Sum: %C", z)!!]</pre>	<pre>AHOY { COIN x = 5, y = 10, z = x + y!! ECHO("Sum: %C", z)!! }</pre>
<pre>AHOY() [COIN x = 10!! COIN y = 20!! COIN sum = x + y!! ECHO("Sum: %C", sum)!!]</pre>	<pre>AHOY(1, 3) [ECHO("Hello, World")!!]</pre>
	<pre>AHOY() [ECHO("Hello, World")!!] COIN add(COIN x, COIN y) [<statement/s>!!]</pre>
	<pre>AHOY() [ECHO("Hello, World")!! BACK 0!!]</pre>

B. Returning Function - This performs a specific computation and returns a value.

Rules for Returning Function

1. A returning function must be defined before the AHOY function and after all the global declarations. Any function definition inside a returning function is invalid.
2. It must start with the return type, followed by a unique identifier, then the parameter list enclosed in parentheses (()), and end with the body enclosed in square brackets ([]).
3. The return type can only be COIN, DIME, PARCH, SCROLL, or BOOL.
4. The parameter list consists of parameter/s. A parameter starts with the data type followed by the identifier. Multiple parameters are allowed and must be separated with commas. The parameter list is optional and can be left empty if not needed.
5. The body must contain a mandatory BACK statement as its final statement to return a value. All other statements are optional. Local declarations are optional and located at the top of the body, before any statement.

Syntax

<pre><return_type> <identifier> (<parameter_list>) [BACK <value>!!]</pre>
<pre><return_type> <identifier> (<parameter_list>) [<statement/s> BACK <value>!!]</pre>
<pre><return_type> <identifier> (<parameter_list>) [<local declaration/s> <statement/s> BACK <value>!!]</pre>

Examples

VALID	INVALID
<pre>COIN add(COIN a, COIN b) [BACK a+b!!]</pre>	<pre>ABYSS addtwo(COIN a, COIN b) [BACK a + b!!]</pre>
<pre>COIN get_ten() [COIN value = 10!! BACK value!!]</pre>	<pre>PARCH get_char() [BACK 123!!]</pre>
<pre>COIN get_max() [COIN max = 100!! BACK 100!!]</pre>	<pre>COIN sum() COIN x, COIN y [BACK x + y!!]</pre>
	<pre>COIN sum(COIN x COIN y)[BACK x + y!!]</pre>
<pre>PARCH get_char(SCROLL text) [PARCH first = 'X'!! BACK first!!]</pre>	<pre>BOOL get_text() [SCROLL text = "test"!!]</pre>
<pre>SCROLL get_name() [SCROLL name = "SeaStack"!! BACK name!!]</pre>	<pre>SCROLL get_number() [COIN num = 42!! BACK 42!!]</pre>
<pre>SCROLL concat_strings(SCROLL first, SCROLL last) [SCROLL full = first & " " & last!! BACK full!!]</pre>	<pre>PARCH get_word() [SCROLL word = "hello"!! BACK 3+5!!]</pre>

C. Non-returning Function - This operates but does not return a value to the caller.

Rules for Non-returning Function

1. A non-returning function must be defined before the AHOY function and after all the global declarations. Any function definition inside a non-returning function is invalid.
2. It must start with the reserved word ABYSS as its return type, indicating it does not return a value. It is followed by a unique identifier, then the parameter list enclosed in parentheses (()), and ends with the body enclosed in square brackets ([]).
3. The parameter list consists of parameter/s. A parameter starts with the data type followed by the identifier. Multiple parameters are allowed and must be separated with commas. The parameter list is optional and can be left empty if not needed.
4. The body must have at least one statement, excluding jump statements. It can end with an optional BACK statement without a return value. Local declarations are optional and located at the top of the body, before any statement.

Syntax

<pre>ABYSS <identifier> (<parameter_list>) [<statement/s>]</pre>
<pre>ABYSS <identifier> (<parameter_list>) [<statement/s> BACK!!]</pre>
<pre>ABYSS <identifier> (<parameter_list>) [<local declaration/s> <statement/s> BACK!!]</pre>

Examples

VALID	INVALID
ABYSS greet() [SCROLL msg = "Hello World"!! ECHO("%S", msg)!!]	ABYSS display() [ECHO("Hello")!! BACK 10 !!]
ABYSS display() [ECHO("SeaStack")!!]	ABYSS display [ECHO("SeaStack")!!]
ABYSS display_sum(COIN a, COIN b) [COIN sum = a + b!! ECHO("Sum: %C", sum)!!]	ABYSS add_numbers(COIN a COIN b) [COIN sum = a + b!! ECHO("The sum is: %C, sum)!!]
ABYSS add_three(COIN x, COIN y, COIN z) [COIN sum = x + y + z!! ECHO("The sum is: %C", sum)!! BACK!!]	ABYSS result() []
ABYSS display_diff(COIN x, COIN y) [COIN diff!! diff = x - y!! ECHO("The difference is: %C", diff)!! BACK!!]	ABYSS display_diff(COIN x, COIN y) [COIN diff!! diff = x - y ECHO("The difference is: %C", diff)!! BACK diff !!]

Rules for Function Calling

1. A function is called by indicating its identifier followed by a pair of parentheses enclosing the arguments, separated by commas, and ending with the statement terminator (!!).
2. The number and types of arguments in a function call must exactly match the function's parameter list and any exceeding or subceeding argument is invalid. The argument can be literals, variables, constants, array elements, structure members, function return values, or expressions.
3. A subfunction can be called inside the AHOY function or another subfunction and must be defined before calling. The AHOY function cannot be called by any function.

Syntax

<code><identifier>()!!</code>
<code><identifier>(<argument1>,<argument2>, ..., <argumentN>)!!</code>

Examples

VALID	INVALID
<code>hello()!!</code>	<code>hello!!</code>
<code>add_numbers(5, 10)!!</code>	<code>add_numbers(5 10)!!</code>
<code>greet_name('A', 20)!!</code>	<code>greet_name{'A', 20}</code>
<code>increment(num)!!</code>	<code>increment(num!!</code>
<code>multiply(5, 3)!!</code>	<code>AHOY()!!</code>

COMMENTS

A comment is an annotation in the source code that helps with understanding the code but is ignored by the system during execution.

Rules for Comments

1. A single-line comment must start with a single tilde symbol (~) and terminates with a newline.
2. A multi-line comment must start with a tilde followed by an open parenthesis ~(. Any following character will be read as a comment until enclosed with a close parenthesis and tilde)~. Unclosed multi-line comments are invalid.
3. Any character following a tilde symbol ~ that is not immediately succeeded by an open parenthesis (is treated as a single-line comment.
4. Comments placed inside a SCROLL literal are considered as a part of the literal and not a comment. Comments in between COIN, DIME, PARCH, or BOOL literals are invalid.

Syntax

~<single-line-comment>
~(<multi-line comment/s>~

Examples

VALID	INVALID
~This is a single line	~
~(This is a multi-line comment It spans multiple lines)~	/* Invalid Symbol */
COIN a = 10!! ~This comment is valid	COIN b = 15 ~Statement not properly terminated
COIN sum = 5 + 10!! ~ valid comment	COIN sum = 5 ~ comment + 10!!

IV. REGULAR DEFINITION

Name	Definition
lowlet	{a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z}
uplet	{A, B, C, D, E, H, L, M, N, P, S}
digit	{0, 1, 2, 3, 4, 5, 6, 7, 8, 9}
nonzero	{1, 2, 3, 4, 5, 6, 7, 8, 9}
alphanumeric	{lowlet, digit}
ASCII	{all printable characters}
whitespace	{ , \t, \n }
arith_op	{+, -, *, /, %, ^}
gen_op	{arith_op, <, >, =, !, &, }
comment	{~}
id_delim	{whitespace, gen_op, (,),], {, }, \$, , }
gen_op_delim	{whitespace, alphanumeric, -, { }
log_op_delim	{gen_op_delim, A, N, ', "}
minus_delim	{whitespace, lowlet, { }
not_delim	{lowlet, A, N, { }
concat_delim	{whitespace, lowlet, (, "}
assign_delim	{log_op_delim, [, !}
back_delim	{whitespace, (, !}
digit_delim	{whitespace, gen_op,),], }, :, , }
bool_delim	{whitespace,),], &, , !, =, , }
parch_delim	{bool_delim, :}

scr_delim	{parch_delim, { }
comma_delim	{assign_delim, @}
colon_delim	{whitespace, lowlet, A, C, E, H, L, +, -}
terminator_delim	{colon_delim, B, D, M, P, S,] }
closecb_delim	{whitespace, gen_op,),], {, &, :, ,}
closep_delim	{whitespace, gen_op,), [,], ,}
openp_delim	{log_op_delim, B, C, D, P, S,), !}
closesb_delim	{whitespace, lowlet, uplet,], !, +, -, ,}
opensb_delim	{closesb_delim, digit, [, ', "}
char_PARCH	{ASCII} - { ' \n \}
char_SCROLL	{ASCII} - { " \n \}
esc_seq_PARCH	{s n t 0 \}
esc_seq_SCROLL	{d n t 0 \}
multi_comment	{ASCII} - {~}

V. REGULAR EXPRESSION

RESERVED WORDS		
Reserved Word	Regular Expression	Token
ABYSS	(A)(B)(Y)(S)(S)	ABYSS
ADRIFT	(A)(D)(R)(I)(F)(T)	ADRIFT
AHOY	(A)(H)(O)(Y)	AHOY
ASK	(A)(S)(K)	ASK
AYE	(A)(Y)(E)	AYE
BACK	(B)(A)(C)(K)	BACK
BOOL	(B)(O)(O)(L)	BOOL
CHART	(C)(H)(A)(R)(T)	CHART
COIN	(C)(O)(I)(N)	COIN
COURSE	(C)(O)(U)(R)(S)(E)	COURSE
DIME	(D)(I)(M)(E)	DIME
DROP	(D)(R)(O)(P)	DROP
DROLOOK	(D)(R)(O)(P)(L)(O)(O)(K)	DROLOOK
ECHO	(E)(C)(H)(O)	ECHO
HAUL	(H)(A)(U)(L)	HAUL
HEAVE	(H)(E)(A)(V)(E)	HEAVE
HOIST	(H)(O)(I)(S)(T)	HOIST
LAND	(L)(A)(N)(D)	LAND
LOCKE	(L)(O)(C)(K)(E)	LOCKE

Reserved Word	Regular Expression	Token
LOOK	(L)(O)(O)(K)	LOOK
MAST	(M)(A)(S)(T)	MAST
NAY	(N)(A)(Y)	NAY
PARCH	(P)(A)(R)(C)(H)	PARCH
SAIL	(S)(A)(I)(L)	SAIL
SCROLL	(S)(C)(R)(O)(L)(L)	SCROLL

RESERVED SYMBOLS		
Reserved Symbol	Regular Expression	Token
+	(+)	+
+#	(+)(#)	+#
+=	(+)(=)	+=
-	(-)	-
-#	(-)(#)	-#
-=	(-)(=)	-=
*	(*)	*
=	()(=)	*=
/	(/)	/
/=	(/)(=)	/=
%	(%)	%
%=	(%)(=)	%=

Reserved Symbol	Regular Expression	Token
^	(^)	^
^=	(^)(=)	^=
=	(=)	=
==	(=)(=)	==
!	(!)	!
!!	(!)(!)	!!
!#	(!)(#)	!#
!=	(!)(=)	!=
<	(<)	<
<=	(<)(=)	<=
>	(>)	>
>=	(>)(=)	>=
&	(&)	&
&&	(&)(&)	&&
	()()	
:	(:)	:
@	(@)	@
\$	(\$)	\$
,	(,)	,
{	({)	{
}	(})	}
(	((	(

Reserved Symbol	Regular Expression	Token
)	(\))	)
[	(\[	[
]	(\])	]

LITERALS		
Literal	Regular Expression	Token
Identifier	(lowlet)(alphanumeric + _ + \) ¹⁹	id
COIN	(- + \)(digit)(digit + \) ¹⁵	COIN-lit
DIME	(- + \)(digit)(digit + \) ¹⁵ (.)(digit)(digit + \) ⁷	DIME-lit
PARCH	(')((char_PARCH) (\)(esc_seq_PARCH))(')	PARCH-lit
SCROLL	(")((char_SCROLL) (\)(esc_seq_SCROLL))+(")	SCROLL-lit
Single-line Comment	(~)(ASCII)*(\n)	single-comment
Multi-line Comment	(~)(\n)(multi_comment)*(\n)~)	multi-comment